

```
# DFS Water-Jug problem

def OccursIn(state, visited):
    return state in visited

def RemoveSeen(children, visited):
    result = []
    for child in children:
        if child not in visited:
            result.append(child)
    return result

def MakePairs(parent, children):
    pairs = []
    for child in children:
        pairs.append((child, parent))
    return pairs

def goalTest(state, target):
    return state[0] == target or state[1] == target

def moveGen(state, cap1, cap2):
    x, y = state
    children = []

    children.append((cap1, y))

    children.append((x, cap2))

    children.append((0, y))

    children.append((x, 0))

    # Pour Jug 1 -> Jug 2
    transfer = min(x, cap2 - y)
    children.append((x - transfer, y + transfer))

    # Pour Jug 2 -> Jug 1
    transfer = min(y, cap1 - x)
    children.append((x + transfer, y - transfer))

    return children

def ReconstructPath(parent, goal):
```

```
path = []
current = goal

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()

print("\nSolution Path:")
for state in path:
    print(state)

cap1 = int(input("Enter Capacity of Jug 1: "))
cap2 = int(input("Enter Capacity of Jug 2: "))
target = int(input("Enter Target Amount: "))

start = (0, 0)

stack = [start]
visited = []
parent = {start: None}

while stack:

    current = stack.pop()

    if OccursIn(current, visited):
        continue

    visited.append(current)
    print("Visited:", current)

    if goalTest(current, target):
        print("\nGoal Reached!")
        ReconstructPath(parent, current)
        break

    children = moveGen(current, cap1, cap2)

    children = RemoveSeen(children, visited)

    pairs = MakePairs(current, children)

    for child, par in pairs:
        if child not in parent:
            parent[child] = par

    stack.append(child)
```

```
else:  
    print("\nNo Solution Exists.")
```

```
Enter Capacity of Jug 1: 3  
Enter Capacity of Jug 2: 7  
Enter Target Amount: 6  
Visited: (0, 0)  
Visited: (0, 7)  
Visited: (3, 4)  
Visited: (3, 0)  
Visited: (0, 3)  
Visited: (3, 3)  
Visited: (0, 6)
```

Goal Reached!

```
Solution Path:  
(0, 0)  
(3, 0)  
(0, 3)  
(3, 3)  
(0, 6)
```

```
# BFS Water-Jug problem  
  
def OccursIn(state, visited):  
    return state in visited  
  
def RemoveSeen(children, visited):  
    result = []  
    for child in children:  
        if child not in visited:  
            result.append(child)  
    return result  
  
def MakePairs(parent, children):  
    pairs = []  
    for child in children:  
        pairs.append((child, parent))  
    return pairs  
  
def goalTest(state, target):  
    return state[0] == target or state[1] == target  
  
def moveGen(state, cap1, cap2):  
    x, y = state  
    children = []
```

```
children.append((cap1, y))
children.append((x, cap2))
children.append((0, y))
children.append((x, 0))

# Pour Jug1 -> Jug2
transfer = min(x, cap2 - y)
children.append((x - transfer, y + transfer))

# Pour Jug2 -> Jug1
transfer = min(y, cap1 - x)
children.append((x + transfer, y - transfer))

return children

def ReconstructPath(parent, goal):
    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()

    print("\nSolution Path:")
    for state in path:
        print(state)

cap1 = int(input("Enter Capacity of Jug1: "))
cap2 = int(input("Enter Capacity of Jug2: "))
target = int(input("Enter Target Amount: "))

start = (0, 0)

queue = [start]
visited = []
parent = {start: None}

while queue:

    current = queue.pop(0)

    if OccursIn(current, visited):
        continue

    visited.append(current)
    print("Visited:", current)
```

```
        if goalTest(current, target):
            print("\nGoal Reached!")
            ReconstructPath(parent, current)
            break

        children = moveGen(current, cap1, cap2)

        children = RemoveSeen(children, visited)

        pairs = MakePairs(current, children)

        for child, par in pairs:

            if child not in parent:
                parent[child] = par

            queue.append(child)

    else:
        print("\nNo Solution Exists.")
```

```
Enter Capacity of Jug1: 3
Enter Capacity of Jug2: 7
Enter Target Amount: 6
Visited: (0, 0)
Visited: (3, 0)
Visited: (0, 7)
Visited: (3, 7)
Visited: (0, 3)
Visited: (3, 4)
Visited: (3, 3)
Visited: (0, 4)
Visited: (0, 6)
```

Goal Reached!

Solution Path:

```
(0, 0)
(3, 0)
(0, 3)
(3, 3)
(0, 6)
```

DFS 8 Puzzle

```
def OccursIn(state, visited):
    return state in visited

def RemoveSeen(children, visited):
    result = []
    for child in children:
```

```
        if child not in visited:
            result.append(child)
    return result

def MakePairs(parent, children):
    pairs = []
    for child in children:
        pairs.append((child, parent))
    return pairs

def goalTest(state, goal):
    return state == goal

def moveGen(state):
    children = []

    state = list(state)

    blank = state.index(0)

    row = blank // 3
    col = blank % 3

    moves = []

    if row > 0:
        moves.append(blank - 3)

    if row < 2:
        moves.append(blank + 3)

    if col > 0:
        moves.append(blank - 1)

    if col < 2:
        moves.append(blank + 1)

    for pos in moves:
        new_state = state.copy()
        new_state[blank], new_state[pos] = new_state[pos], new_state[blank]
        children.append(tuple(new_state))

    return children

def ReconstructPath(parent, goal):
    path = []
```

```
        current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()

    print("\nSolution Path:\n")

    for state in path:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()

    print("Enter Initial State")
    initial = tuple(map(int, input().split()))

    print("Enter Goal State")
    goal = tuple(map(int, input().split()))

    stack = [initial]
    visited = []
    parent = {initial: None}

    while stack:

        current = stack.pop()

        if OccursIn(current, visited):
            continue

        visited.append(current)

        if goalTest(current, goal):
            print("Goal Reached!")
            ReconstructPath(parent, current)
            break

        children = moveGen(current)

        children = RemoveSeen(children, visited)

        pairs = MakePairs(current, children)

        for child, par in pairs:

            if child not in parent:
                parent[child] = par
```

```
        stack.append(child)

    else:
        print("No Solution Exists")
```

```
Enter Initial State
1 2 3 4 5 6 7 0 8
Enter Goal State
1 2 3 4 5 6 7 8 0
Goal Reached!
```

Solution Path:

```
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

```
# BFS 8puzzle

def OccursIn(state, visited):
    return state in visited

def RemoveSeen(children, visited):
    result = []
    for child in children:
        if child not in visited:
            result.append(child)
    return result

def MakePairs(parent, children):
    pairs = []
    for child in children:
        pairs.append((child, parent))
    return pairs

def goalTest(state, goal):
    return state == goal

def moveGen(state):
    children = []
```



```
state = list(state)
blank = state.index(0)

row = blank // 3
col = blank % 3

moves = []

if row > 0:
    moves.append(blank - 3)

if row < 2:
    moves.append(blank + 3)

if col > 0:
    moves.append(blank - 1)

if col < 2:
    moves.append(blank + 1)

for pos in moves:
    new_state = state.copy()
    new_state[blank], new_state[pos] = new_state[pos], new_state[blank]
    children.append(tuple(new_state))

return children

def ReconstructPath(parent, goal):
    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()

    print("\nSolution Path:\n")

    for state in path:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()

print("Enter Initial State")
initial = tuple(map(int, input().split()))

print("Enter Goal State")
```

```
goal = tuple(map(int, input().split()))

queue = [initial]
visited = []
parent = {initial: None}

while queue:

    current = queue.pop(0)

    if OccursIn(current, visited):
        continue

    visited.append(current)

    if goalTest(current, goal):
        print("Goal Reached!")
        ReconstructPath(parent, current)
        break

    children = moveGen(current)

    children = RemoveSeen(children, visited)

    pairs = MakePairs(current, children)

    for child, par in pairs:

        if child not in parent:
            parent[child] = par

        queue.append(child)

else:
    print("No Solution Exists")
```

```
Enter Initial State
1 2 3 4 5 6 7 0 8
Enter Goal State
1 2 3 4 5 6 7 8 0
Goal Reached!
```

Solution Path:

```
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Start coding or [generate](#) with AI.